

15.2.3 Processing files

Our original problem of answering whether a file has more than 40,000 elements is now easy to solve. But so far we've just been transforming pure streams. Luckily, we can just as easily use a file to drive a `Process`. And instead of generating a `Stream` as the result, we can accumulate what the `Process` emits similar to what `foldLeft` does on `List`.

Listing 15.3 Using `Process` with files instead of streams

```
def processFile[A,B](f: java.io.File,
                    p: Process[String, A],
                    z: B)(g: (B, A) => B): IO[B] = IO {
  @annotation.tailrec
  def go(ss: Iterator[String], cur: Process[String, A], acc: B): B =
    cur match {
      case Halt() => acc
      case Await(recv) =>
        val next = if (ss.hasNext) recv(Some(ss.next))
                   else recv(None)
        go(ss, next, acc)
      case Emit(h, t) => go(ss, t, g(acc, h))
    }
  val s = io.Source.fromFile(f)
  try go(s.getLines, p, z)
  finally s.close
}
```

We can now solve the original problem with the following:

```
processFile(f, count |> exists(_ > 40000), false)(_ || _)
```



EXERCISE 15.9

Write a program that reads degrees Fahrenheit as `Double` values from a file, one value per line, sends each value through a process to convert it to degrees Fahrenheit, and writes the result to another file. Your program should ignore blank lines in the input file, as well as lines that start with the `#` character. You can use the function `toCelsius`.

```
def toCelsius(fahrenheit: Double): Double =
  (5.0 / 9.0) * (fahrenheit - 32.0)
```

15.3 An extensible process type

Our existing `Process` type implicitly assumes an *environment* or *context* containing a single stream of values. Furthermore, the *protocol* for communicating with the driver is also fixed. A `Process` can only issue three instructions to the driver—`Halt`, `Emit`, and

Await—and there’s no way to extend this protocol short of defining a completely new type. In order to make `Process` extensible, we’ll parameterize on the protocol used for issuing requests of the driver. This works in much the same way as the `Free` type we covered in chapter 13.

Listing 15.4 An extensible `Process` type

```

trait Process[F[_],O]

object Process {
  case class Await[F[_],A,O] (
    req: F[A],
    recv: Either[Throwable, A] => Process[F,O]
  ) extends Process[F,O]

  case class Emit[F[_],O] (
    head: O,
    tail: Process[F,O]
  ) extends Process[F,O]

  case class Halt[F[_],O] (err: Throwable) extends Process[F,O]

  case object End extends Exception
  case object Kill extends Exception
}

```

The `recv` function now takes an `Either` so we can handle errors.

Halt due to `err`, which could be an actual error or `End` indicating normal termination.

An `Exception` that indicates normal termination. This allows us to use Scala’s exception mechanism for control flow.

An `Exception` that indicates forceful termination. We’ll see how this is used later.

Unlike `Free[F,A]`, a `Process[F,O]` represents a *stream* of `O` values (`O` for output) produced by (possibly) making external requests using the protocol `F` via `Await`. The `F` parameter serves the same role here in `Await` as the `F` parameter used for `Suspend` in `Free` from chapter 13.

The important difference between `Free` and `Process` is that a `Process` can request to `Emit` values multiple times, whereas `Free` always contains one answer in its final `Return`. And instead of terminating with `Return`, the `Process` terminates with `Halt`.

To ensure resource safety when writing processes that close over some resource like a file handle or database connection, the `recv` function of `Await` takes an `Either[Throwable,A]`. This lets `recv` decide what should be done if there’s an error while running the request `req`.⁵ We’ll adopt the convention that the `End` exception indicates that there’s no more input, and `Kill` indicates the process is being forcibly terminated and should clean up any resources it’s using.⁶

⁵ The `recv` function should be trampolined to avoid stack overflows by returning a `TailRec[Process[F,O]]`, but we’ve omitted this detail here for simplicity.

⁶ There are some design decisions here—we’re using exceptions, `End` and `Kill`, for control flow, but we could certainly choose to indicate normal termination with `Option`, say with `Halt[F[_],O] (err: Option[Throwable])`.

The `Halt` constructor picks up a *reason* for termination in the form of a `Throwable`. The reason may be `End`, indicating normal termination due to exhausted input; `Kill`, indicating forcible termination; or some other error (note that `Exception` is a subtype of `Throwable`).

This new `Process` type is more general than the previous `Process`, which we'll refer to from now on as a “single-input `Process`” or a “`Process1`”, and we can represent a single-input `Process` as a special instance of this generalized `Process` type. We'll see how this works in section 15.3.3.

First, note that a number of operations are defined for `Process` *regardless* of the choice of `F`. We can still define `++` (append), `map`, and `filter` for `Process`, and the definitions are almost identical to before. Here's `++` (see chapter code for other functions, including `repeat`, `map`, and `filter`), which we define in terms of a more general function, `onHalt`.

Listing 15.5 The `onHalt` and `++` functions

Uses the
helper
function
`Try`.

```
def onHalt(f: Throwable => Process[F,O]): Process[F,O] = this match {
  case Halt(e) => Try(f(e))
  case Emit(h, t) => Emit(h, t.onHalt(f))
  case Await(req,recv) => Await(req, recv andThen (_.onHalt(f)))
}
def ++(p: => Process[F,O]): Process[F,O] =
  this.onHalt {
    case End => p
    case err => Halt(err)
  }
```

Consult `p` only on
normal termination.

Otherwise, keep
the current error.

A call to `p.onHalt(f)` replaces the `e` inside the `Halt(e)` at the end of `p` with `f(e)`, allowing us to extend a process with further logic and giving us access to the reason for termination. The definition uses the helper function `Try` with a capital `T`, which safely evaluates a `Process`, catching any exceptions and converting them to `Halt` (see the following code). This is important for resource safety. In general, our goal is to catch and deal with all exceptions, rather than placing that burden on users of our library. Luckily, there are just a few key combinators that can generate exceptions. As long as we ensure these combinators are exception-safe, we'll be able to guarantee the resource safety of all programs that use `Process`. The `++` function is defined in terms of `onHalt`—so long as the first `Process` terminates normally, we continue with the second process; otherwise, we re-raise the error.

Here's the helper function `Try`:

```
def Try[F[_],O](p: => Process[F,O]): Process[F,O] =
  try p
  catch { case e: Throwable => Halt(e) }
```

We'll also introduce the helper function `await`, which is a curried version of the `Await` constructor for better type inference: